CURSO DE ESPECIALIZACIÓN EN
INTELIGENCIA ARTIFICIAL Y BIG DATA

FASE 1 — DOCUMENTACIÓN Y DISEÑO TÉCNICO

FarmaPredict

Predicción de desabastecimientos de medicamentos
y recomendación de sustitutivos

Integrante	Rol principal
Alejandro Ronda	Data / Platform
Francisco Antonio Tortosa	Machine Learning
Mohamed Ayman Oumghar Seghrouchni	BI / Dashboard

Grupo 9 · Semestre 2609
Repositorio: github.com/arondax/farmapredict

Contenido

- 1. Título del proyecto e integrantes
- 2. Resumen ejecutivo
- 3. Descripción funcional del sistema
- 4. Alcance definitivo (IN / OUT)
- 5. Arquitectura del sistema
- 6. Tecnologías seleccionadas
- 7. Modelo de datos
- 8. Datos
- 9. Ética, legalidad y accesibilidad
- 10. Backlog inicial
- 11. Plan de trabajo
- 12. Estrategia de desarrollo
- 13. Riesgos técnicos actualizados
- 14. Repositorio del proyecto

1. Título del proyecto e integrantes

Nombre del proyecto: FarmaPredict - Predicción de desabastecimientos de medicamentos y recomendación de sustitutivos.

Miembro	Rol	Responsabilidad principal
Alejandro Ronda	Data / Platform	Ingesta CIMA, PostgreSQL, Airflow, FastAPI, Docker Compose e integración.
Francisco Antonio Tortosa	Machine Learning	Definición de etiqueta, baseline, XGBoost, evaluación y experimentación.
Mohamed Ayman Oumghar Seghrouchni	BI / Dashboard	Streamlit, KPIs, visualización, comparador de sustitutivos y documentación.

2. Resumen ejecutivo

Las oficinas de farmacia disponen de información pública sobre problemas de suministro, pero esta información es principalmente reactiva y se encuentra repartida entre distintas fuentes. FarmaPredict propone una plataforma de datos e inteligencia artificial que centraliza información de medicamentos, construye un histórico propio mediante snapshots, estima el riesgo de que un medicamento presente un problema de suministro en los próximos 30 días y propone posibles sustitutivos equivalentes mediante reglas basadas en principio activo, dosis y forma farmacéutica. La solución se expondrá mediante una API REST y un dashboard orientado a farmacéuticos y gestores de compras. El valor principal del sistema es permitir una actuación anticipada: priorizar revisión de stock, detectar medicamentos con mayor riesgo y consultar alternativas informativas antes de que el problema llegue al mostrador. El sistema no realiza prescripción clínica y la decisión final sobre cualquier sustitución corresponde al profesional farmacéutico.

3. Descripción funcional del sistema

3.1 Qué hace la aplicación

- Obtiene periódicamente datos públicos de medicamentos e incidencias de suministro.
- Normaliza y persiste la información en una base de datos relacional.
- Genera variables derivadas y ejecuta un modelo de clasificación de riesgo a 30 días.
- Devuelve una probabilidad y una categoría de riesgo (por ejemplo: Bajo / Medio / Alto; umbrales por validar).
- Busca medicamentos potencialmente equivalentes aplicando reglas de coincidencia definidas por el equipo.
- Expone los resultados mediante FastAPI y los visualiza mediante Streamlit.

3.2 Flujo general de uso

1. El pipeline de ingesta obtiene los datos de la fuente pública y registra el resultado de la carga.
2. Los datos se limpian, normalizan y almacenan en PostgreSQL junto con snapshots históricos.
3. El pipeline de features prepara las variables necesarias para entrenamiento o inferencia.
4. El modelo calcula el riesgo de desabastecimiento para cada medicamento elegible.
5. El usuario accede al dashboard, busca un medicamento o revisa el listado de mayor riesgo.
6. La aplicación consulta la API y muestra riesgo, fecha de cálculo y, cuando aplica, sustitutivos candidatos.

3.3 Casos principales de usuario

Caso	Actor	Resultado esperado
CU01 Consultar riesgo	Farmacéutico / gestor	Obtiene nivel de riesgo y probabilidad para un medicamento.
CU02 Consultar sustitutivos	Farmacéutico	Obtiene alternativas candidatas según reglas de equivalencia.
CU03 Revisar alertas	Gestor de compras	Visualiza medicamentos priorizados por riesgo.
CU04 Actualizar datos	Sistema / equipo	Ejecuta ingesta, registra carga y conserva snapshot.

3.4 Qué puede y qué no puede hacer

Puede	No puede
Estimar riesgo con las variables disponibles.	Garantizar que se producirá o no un desabastecimiento.
Proponer sustitutivos candidatos mediante reglas.	Prescribir ni sustituir automáticamente un medicamento.
Trabajar con datos públicos/agregados y sintéticos.	Utilizar datos reales de pacientes o farmacias en el MVP.
Mostrar trazabilidad básica de fecha y versión del modelo.	Sustituir el criterio profesional del farmacéutico.

4. Alcance definitivo (IN / OUT)

4.1 Incluido en el MVP

- Ingesta programada desde AEMPS/CIMA mediante Python y Airflow.
- Almacenamiento relacional en PostgreSQL y creación de histórico mediante snapshots.
- Baseline de referencia y clasificador XGBoost para riesgo a 30 días.
- Recomendador por reglas sobre principio activo, dosis y forma farmacéutica.
- API REST con FastAPI.
- Dashboard con Streamlit y Plotly.
- Entorno reproducible con Docker Compose.
- Tests básicos con PyTest y documentación de ejecución.

4.2 Funcionalidades opcionales

- Previsión estacional por ATC con Prophet, solo si existe una fuente adecuada y el MVP está completo.
- Ordenación semántica de alternativas mediante embeddings y Qdrant.
- Uso de PySpark únicamente si el volumen real lo justifica.
- Simulador de pedidos con datos sintéticos para demostración.

4.3 Fuera de alcance

- Integración con robots de almacenamiento, ERP o receta electrónica.
- Aplicaciones móviles nativas.
- Diagnóstico médico o prescripción.
- Datos reales identificables de pacientes o farmacias.
- Autenticación multiusuario y gestión avanzada de permisos.
- Despliegue productivo en cloud.

- Reentrenamiento automático y monitorización MLOps completa.

5. Arquitectura del sistema

La arquitectura propuesta separa adquisición, persistencia, modelado, exposición y visualización. Esta separación permite probar cada componente de forma independiente, mantener una trazabilidad básica de los datos y reducir el acoplamiento entre las distintas partes del sistema.

A alto nivel, la solución parte de las fuentes públicas de datos, ejecuta un proceso de ingesta y limpieza, persiste la información en PostgreSQL, calcula features para el modelo, genera predicciones y publica el resultado mediante una API consumida por un dashboard en Streamlit.

Componente	Responsabilidad
Fuentes	Aportan catálogo, incidencias y, si es viable, demanda agregada.
Airflow + Python	Orquestan descarga, limpieza, validaciones y carga.
PostgreSQL	Mantiene catálogo, histórico, features y resultados.
Servicio ML	Entrena/evalúa modelos y genera inferencias.
Recomendador	Aplica reglas de equivalencia definidas por el equipo.
FastAPI	Expone endpoints de riesgo, sustitutivos y estado del servicio.
Streamlit	Interfaz de consulta y dashboard.
Docker Compose	Levanta los servicios de forma reproducible.

6. Tecnologías seleccionadas

Tecnología	Uso	Justificación	Observaciones
Python	ETL, ML, API y lógica	Unifica el stack y dispone de ecosistema maduro para datos y ML.	-
PostgreSQL	Persistencia	Permite trabajar con datos estructurados y relaciones claras entre medicamentos, incidencias y predicciones.	Base de datos principal del MVP.
Apache Airflow	Orquestación	Permite programar, observar y reintentar pipelines.	Cron como plan B si la complejidad supera el valor aportado.
XGBoost	Modelo principal	Adecuado para clasificación tabular y permite tratar desbalanceo.	Regresión logística como baseline.
scikit-learn	Baseline y métricas	Implementación rápida de modelos de referencia y evaluación.	-
FastAPI	Backend/API	Ligero, tipado y bien integrado con Python.	Incluye documentación automática de endpoints.
Streamlit	Frontend/dashboard	Reduce tiempo de desarrollo para un prototipo analítico.	Frontend web dedicado queda fuera del MVP.
Plotly	Gráficos	Visualización interactiva integrada con Streamlit.	-
Docker Compose	Entorno	Reproducibilidad local multi-servicio.	Despliegue cloud fuera de alcance.
PyTest	Testing	Automatización de pruebas unitarias e integración básica.	-

7. Modelo de datos

El siguiente modelo es una propuesta de diseño para la Fase 1. Los nombres y campos concretos deberán ajustarse a la respuesta real de CIMA una vez se cierre el análisis de los endpoints. Se presenta con un formato visual similar a draw.io para reflejar mejor cómo podría documentarse en la entrega final.

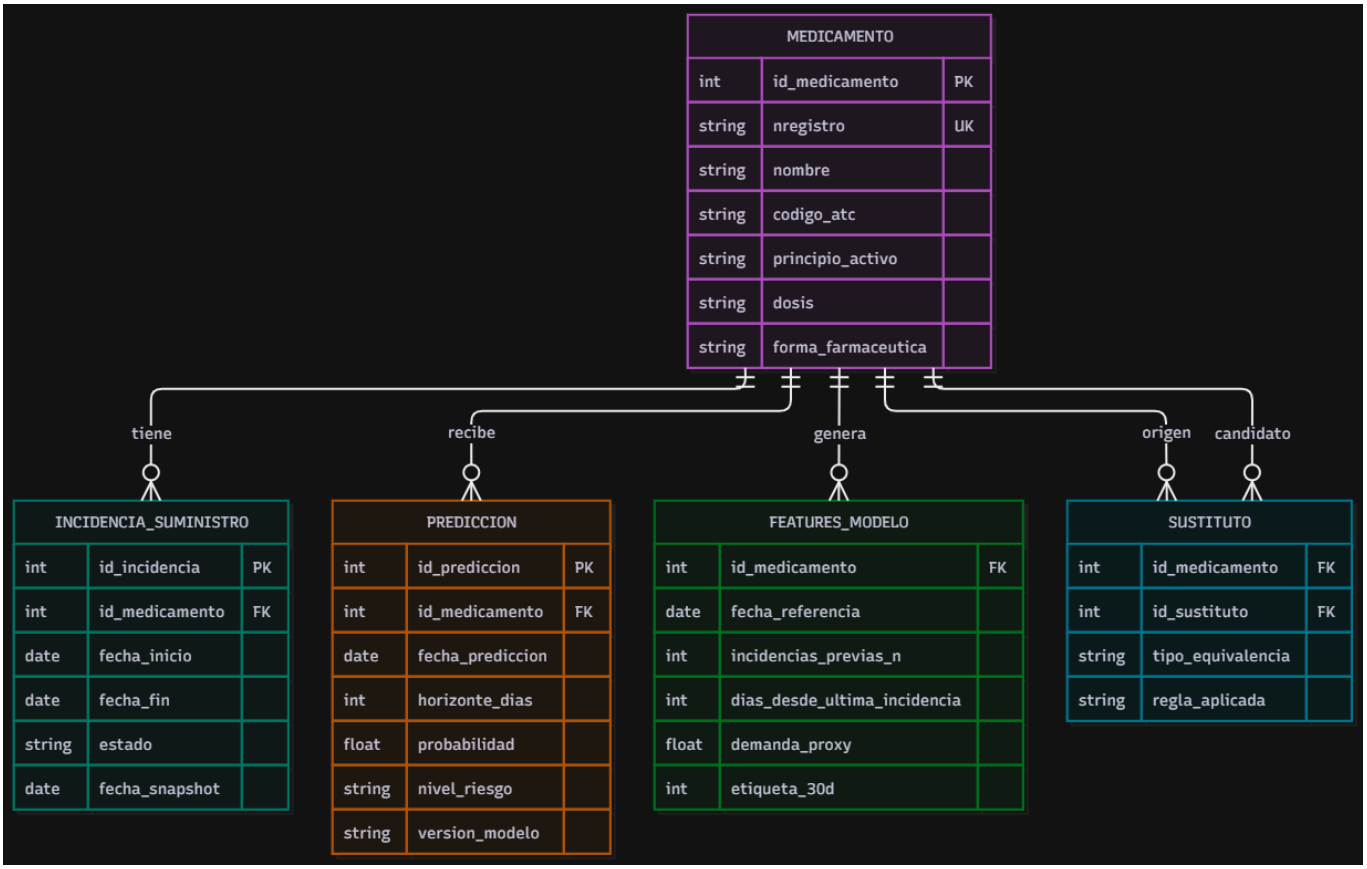


Figura 1. Modelo semantico de las tablas propuestas

7.1 Descripción de entidades

Entidad	Finalidad	Clave
MEDICAMENTO	Catálogo normalizado de medicamentos.	id_medicamento; nregistro como identificador externo si aplica.
INCIDENCIA_SUMINISTRO	Episodios o snapshots de problemas de suministro.	id_incidencia + FK a medicamento.
PREDICCION	Resultados históricos de inferencia.	id_prediccion + FK a medicamento.
SUSTITUTO	Relación entre medicamento origen y candidato alternativo.	Clave compuesta origen-candidato.
FEATURES_MODELO	Variables por medicamento y fecha de referencia.	medicamento + fecha.

7.2 Diccionario mínimo de variables del modelo

Variable	Tipo	Descripción	Estado
codigo_atc	Categórica	Clasificación terapéutica del medicamento.	Prevista
principio_activo	Categórica	Principio activo normalizado.	Prevista
forma_farmaceutica	Categórica	Forma de administración/presentación.	Prevista
incidencias_previas_n	Numérica	Número de incidencias previas dentro de una ventana temporal.	Por diseñar
dias_desde_ultima_incidencia	Numérica	Recencia del último problema de suministro.	Por diseñar
demanda_proxy	Numérica	Variable agregada procedente del Ministerio, si existe granularidad suficiente.	Condicionada
etiqueta_30d	Binaria	1 si existe problema de suministro dentro de los 30 días posteriores.	Sujeta a datos CIMA

8. Datos

Fuente	Tipo	Volumen / frecuencia	Naturaleza	Riesgos / observaciones
AEMPS CIMA - catálogo e incidencias	JSON + texto	≈15.000 medicamentos indicado en Fase 0; frecuencia real por confirmar	Real / pública	Histórico y estructura exacta del endpoint pendientes de verificar.
Ministerio de Sanidad - facturación SNS	Tabular agregada	Granularidad y periodicidad pendientes de confirmar	Real / pública	Puede no tener detalle suficiente para servir como proxy de demanda.
Generador Faker + NumPy	Tabular	Configurable	Sintético	Solo para validar pipeline, dashboard y volumen; no para demostrar capacidad predictiva.

8.1 Calidad esperada y validaciones

- Control de duplicados por identificador de medicamento y fecha de snapshot.
- Validación de campos obligatorios antes de cargar en tablas finales.
- Registro de filas rechazadas y motivo.
- Normalización de formatos de fecha y categorías.
- Seguimiento de valores nulos en variables relevantes.
- Comprobación de cambios de esquema en la API.

8.2 Licencias y reutilización

En esta fase se da por revisado el uso de las fuentes públicas previstas para el proyecto. La documentación técnica, el README y la memoria citarán siempre la fuente original. El repositorio contendrá código, documentación y datos sintéticos o de ejemplo, pero no necesariamente copias completas de los conjuntos descargados cuando no sea necesario redistribuirlos. La ingesta automatizada se utilizará con fines académicos, respetando las condiciones de reutilización de cada fuente, manteniendo la trazabilidad del origen y evitando incorporar información identificable de personas.

9. Ética, legalidad y accesibilidad

Área	Decisión de diseño
Datos personales / PII	El MVP no necesita nombres, DNI, historias clínicas ni recetas individuales.
RGPD	Diseño basado en datos públicos, agregados o sintéticos; se evitará incorporar información identificable.
Sesgo	Para reducir sesgos y proteger la interpretación del modelo, se priorizarán variables estructuradas y categóricas (por ejemplo, código ATC, principio activo y forma farmacéutica), se revisará el rendimiento por subgrupos y se evitará presentar una única métrica global sin contexto.
Interpretación del resultado	La predicción debe entenderse como una ayuda a la decisión y no como una certeza. La interfaz mostrará la fecha de cálculo, el nivel de riesgo y un mensaje breve que recuerde que el resultado puede cambiar cuando entren nuevos datos.
Sustitutivos	Resultado informativo; no constituye prescripción ni reemplaza el criterio del farmacéutico.
Accesibilidad	En este punto, la accesibilidad se refiere a la interfaz del sistema (si existe frontend), no al acceso público a las fuentes. Por ello se prevé usar texto además de colores, contraste suficiente, navegación simple y mensajes claros.

10. Backlog inicial

El backlog inicial es la lista priorizada de trabajo del proyecto. En este contexto puede incluir tanto historias orientadas al usuario final (por ejemplo, farmacéutico o gestor) como historias técnicas necesarias para que la solución exista y funcione. Es decir, no todas las entradas tienen por qué representar a un usuario externo; algunas describen trabajo del propio equipo de datos, ML o plataforma.

ID	Historia de usuario	Criterios de aceptación (resumen)

ID	Historia de usuario	Criterios de aceptación (resumen)
HU01	Como farmacéutico quiero buscar un medicamento para consultar su riesgo.	La búsqueda devuelve coincidencias; al seleccionar uno se muestra riesgo, fecha y versión; si no hay datos se informa.
HU02	Como farmacéutico quiero consultar sustitutivos candidatos para actuar ante una incidencia.	Se muestran candidatos que cumplen las reglas configuradas; se incluye aviso de carácter informativo.
HU03	Como gestor quiero ver medicamentos con riesgo alto para priorizar revisión de stock.	Existe una vista ordenable/filtrable por nivel de riesgo y medicamento.
HU04	Como equipo de datos quiero ejecutar una ingesta programada de CIMA.	La ejecución registra inicio/fin, filas procesadas y estado; los fallos quedan trazados.
HU05	Como equipo de datos quiero conservar snapshots para construir histórico.	Cada ejecución válida queda asociada a una fecha y no sobrescribe silenciosamente el histórico.
HU06	Como equipo ML quiero disponer de un dataset reproducible de features.	Las features se generan con una fecha de referencia y reglas documentadas.
HU07	Como equipo ML quiero comparar XGBoost con una línea base.	Se calcula PR-AUC para ambos bajo la misma partición y se documenta el resultado.
HU08	Como consumidor de la API quiero consultar riesgo por identificador de medicamento.	El endpoint devuelve respuesta estructurada y códigos HTTP coherentes.
HU09	Como equipo técnico quiero comprobar que los servicios están disponibles.	Existe endpoint de salud y Docker Compose levanta los servicios mínimos.
HU10	Como usuario quiero conocer cuándo se actualizaron los datos.	El dashboard muestra fecha/hora de la última carga válida.

11. Plan de trabajo

El siguiente cronograma se deja como plantilla rellenable para acordarlo con el equipo. La idea es completar semanas, hitos, responsables y dependencias una vez se revisen prioridades y disponibilidad real.

Semana / sprint	Objetivo / tareas	Responsable principal	Dependencias / hito
Semana 1	_____	_____	_____
Semana 2	_____	_____	_____
Semana 3	_____	_____	_____
Semana 4	_____	_____	_____
Semana 5	_____	_____	_____
Semana 6	_____	_____	_____

11.1 Hitos principales (rellenable)

- H1 - _____
- H2 - _____
- H3 - _____
- H4 - _____
- H5 - _____

12. Estrategia de desarrollo

12.1 Flujo Git

Se propone un flujo ligero basado en ramas de funcionalidad. La rama main contendrá únicamente versiones integradas y revisadas; develop actuará como rama de integración durante el desarrollo.

Rama	Uso
main	Estado estable / entregable.
develop	Integración de funcionalidades antes de pasar a main.

Rama	Uso
feature/data-*	Ingesta, esquema y transformaciones.
feature/ml-*	Features, modelos y evaluación.
feature/api-*	Backend y contratos REST.
feature/dashboard-*	Interfaz Streamlit y visualizaciones.

12.2 Integración y validación

- Todo cambio parte de una issue con responsable.
- No se integra directamente en main.
- Cada feature se incorpora mediante Pull Request.
- La PR requiere revisión de al menos otra persona cuando sea viable.
- Antes de mergear se ejecutan tests afectados y una comprobación funcional mínima.
- Las decisiones de diseño relevantes se documentan en issues, PR o README.

12.3 Testing básico

Nivel	Ejemplos
Unitario	Normalización de campos, reglas de sustitutivos, funciones de features.
API	Códigos HTTP, esquema de respuesta, casos sin datos.
Datos	Duplicados, nulos críticos, integridad referencial y cambios de esquema.
Integración	Ingesta → PostgreSQL → inferencia → API → dashboard.
Reproducibilidad	Levantar servicios mínimos desde un entorno limpio con Docker Compose.

13. Riesgos técnicos actualizados

Riesgo	Impacto	Mitigación
CIMA no ofrece histórico suficiente	Alto	Acumular snapshots desde el inicio; adaptar objetivo o categorías de salida si el histórico es limitado.
Cambios, caídas o límites de API	Alto	Caché local, snapshots descargados y validación de esquema.
Datos de demanda sin granularidad suficiente	Medio	Tratar la demanda como ampliación; usar sintético solo para demostrar pipeline.
Desbalanceo de clases / modelo no supera baseline	Alto	Pesos de clase, PR-AUC, revisión de horizonte/features; documentar resultado negativo si procede.
Alcance excesivo	Alto	Bloquear ampliaciones hasta completar MVP end-to-end.
Complejidad de Airflow/Docker en varios equipos	Medio	Servicios mínimos; cron como plan B de orquestación.
Equivalencia de sustitutivos mal interpretada	Alto	Reglas explícitas, revisión del equipo y aviso de uso informativo.
Licencias/condiciones de reutilización no verificadas	Medio	Revisión oficial antes de automatizar o redistribuir datos.

14. Repositorio del proyecto

Repositorio previsto: https://github.com/arondax/farmapredict

Estructura mínima propuesta:

```
repo/
├── docs/
│   ├── NF2_Documentacion_y_Diseno_Tecnico.pdf
│   └── modelo_datos.png
├── src/
│   ├── ingestion/
│   ├── ml/
```

```
│   ├── api/
│   └── dashboard/
├── data/
│   └── .gitkeep
├── tests/
├── environment/
│   └── docker-compose.yml
├── README.md
└── .gitignore
```

14.1 Contenido mínimo del README

- Descripción y objetivo del proyecto.
- Integrantes y roles.
- Arquitectura resumida.
- Tecnologías principales.
- Instrucciones de preparación y ejecución, aunque algunas partes aún estén en construcción.
- Descripción de carpetas y política para datos no versionados.